



CSCI 232

232 Algorithms & Data Structures

LECTURE: 10

J. L. Pach

TODAY'S AGENDA

- ✂ What is an Algorithm?
- ✂ NP vs P
- ✂ Algorithm Notation Systems
- ✂ The RAM Model — Primitive Operations
- ✂ Counting Primitive Operations
- ✂ From $T(n)$ to Asymptotic Notation
- ✂ Array Indexing — The AGU Rule
- ✂ Algorithm Correctness — arrayMax
- ✂ Best Case, Worst Case, Average Case

ALGORITHM

ALGORITHM — DEFINITION

Informally, an **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**. An algorithm is thus a sequence of computational steps that transform the input into the output.

We can also view an algorithm as a tool for solving a well-specified **computational problem**. The statement of the problem specifies in general terms the desired input/output relationship. The algorithm describes a specific computational procedure for achieving that input/output relationship.

ALGORITHM — THE SORTING PROBLEM

For example, we might need to sort a sequence of numbers into nondecreasing order. Here is how we formally define the sorting problem:

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$

Output: A permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$

For example, given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.

ALGORITHM — INSTANCE AND CORRECTNESS

Such an input sequence is called an **instance** of the sorting problem. In general, an **instance of a problem** consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.

An algorithm is said to be **correct** if, for every input instance, it halts with the correct output. We say that a correct algorithm **solves** the given computational problem.

An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.

PRACTICAL APPLICATIONS OF ALGORITHMS

- ✂ The **Human Genome Project** — identifying all 100,000 genes in human DNA, determining the sequences of the 3 billion chemical base pairs.
- ✂ The **Internet** — clever algorithms enable sites to manage and manipulate large volumes of data quickly.
- ✂ **Electronic commerce** — depends on the privacy of personal information such as credit card numbers, passwords, and bank statements.
- ✂ **Manufacturing** — allocating scarce resources in the most beneficial way.
- ✂ **Navigation** — finding the shortest route from one intersection to another on a road map.

TWO CHARACTERISTICS OF INTERESTING ALGORITHMIC PROBLEMS

1. They have **many candidate solutions**, the overwhelming majority of which do not solve the problem at hand. Finding one that does, or one that is "best," can present quite a challenge.
2. They have **practical applications**. Of the problems in the above list, finding the shortest path provides the easiest examples. A transportation firm has a financial interest in finding shortest paths because taking shorter paths results in lower labor and fuel costs.

DATA STRUCTURES

A data structure is a way to store and organize data in order to facilitate access and modifications.

No single data structure works well for all purposes, and so it is important to know the **strengths and limitations** of several of them.

NP VS P

HARD PROBLEMS

- ✂ Most of this course is about **efficient algorithms**. Our usual measure of efficiency is speed — how long an algorithm takes to produce its result.
- ✂ There are some problems, however, for which **no efficient solution is known**.
- ✂ An interesting subset of these problems are known as **NP-complete** (nondeterministic polynomial-time complete).
- ✂ Although no efficient algorithm for an NP-complete problem has ever been found, **nobody has ever proven** that an efficient algorithm for one cannot exist.

NP VS P

- ✂ **P problems** — can be **solved** in polynomial time.
- ✂ **NP problems** — those for which the correctness of a given solution can be **verified** in polynomial time.

Imagine having a riddle. Finding the solution to this riddle might be very difficult and time-consuming. However, if someone gives you a solution, you can quickly verify whether it is correct.

This is somewhat analogous to NP problems: **finding a solution can be hard, but verifying a given solution is easy.**

NP-COMPLETE EXAMPLE — TRAVELING SALESMAN

Consider a delivery company with a central depot. Each day, it loads up each delivery truck at the depot and sends it around to deliver goods to several addresses. At the end of the day, each truck must end up back at the depot.

To reduce costs, the company wants to select an order of delivery stops that yields the **lowest overall distance** traveled by each truck.

This problem is the well-known **traveling-salesman problem** — it is NP-complete. It has no known efficient algorithm. Under certain assumptions, however, we know of efficient algorithms that give an overall distance which is **not too far above the smallest possible**.

ALGORITHM

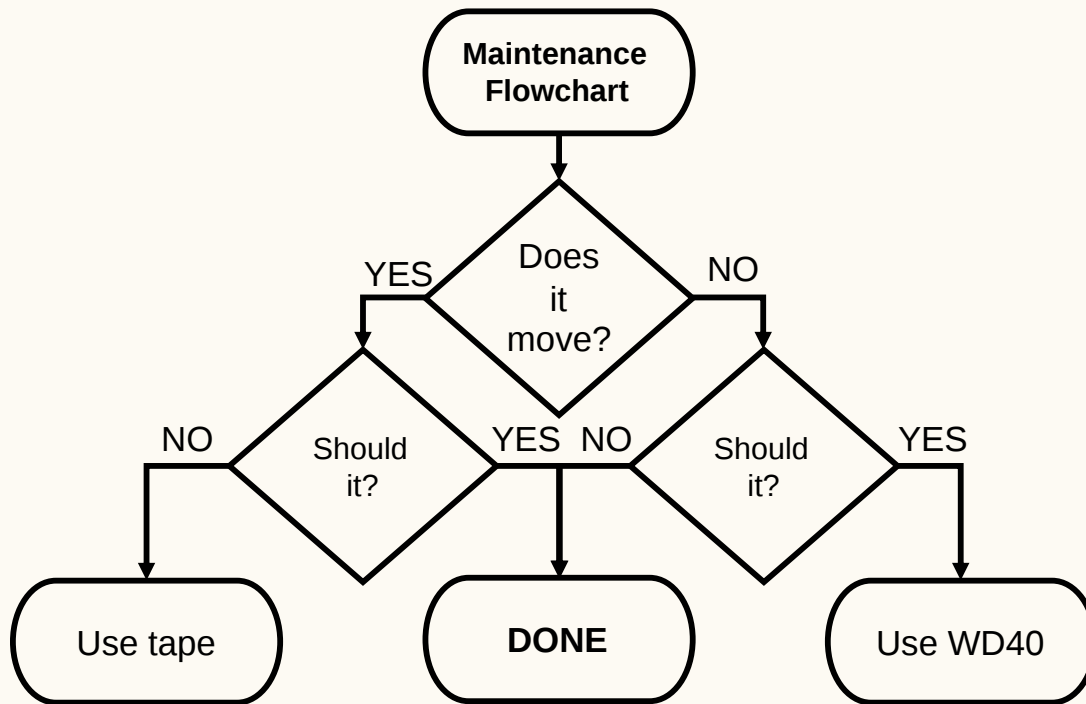
Notation Systems

AN ALGORITHM CAN BE REPRESENTED IN THREE WAYS

- ✂ **Flowchart** — breaks down the algorithm into elementary control blocks: start, stop, data input, output, conditional statements.
- ✂ **Bullet point description** — individual steps described verbally using numbered bullet points.
- ✂ **Pseudocode** — resembles a programming language but uses only keywords such as `if`, `for`, `while`, `return`. Today Python is most popular for describing programming concepts due to its simplicity.

FLOWCHART

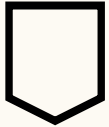
A flowchart is a **graphical representation** of an algorithm. It uses shapes to represent different types of operations and arrows to indicate the flow of control between them.



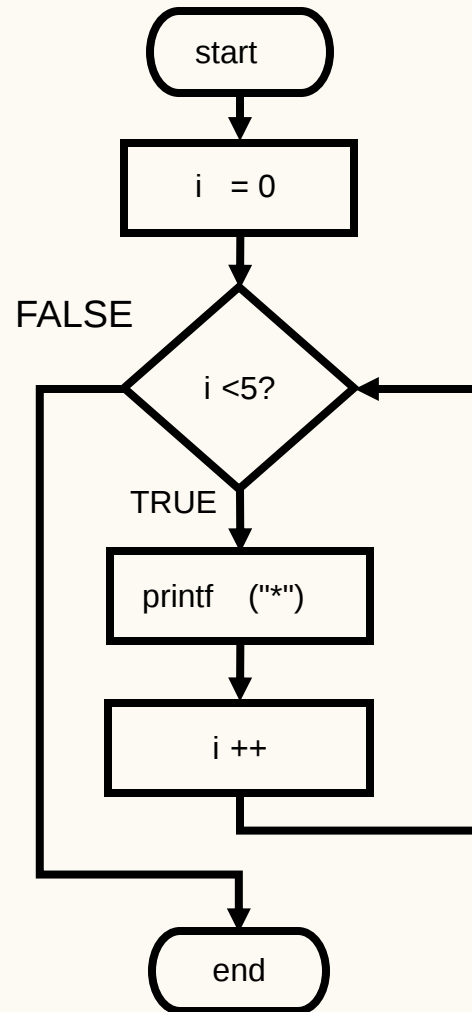
FLOWCHART SYMBOLS 1/2

Shape	Name	Description
	Flowline/Arrowhead	Shows the process's order of operation. A line coming from one symbol and pointing at another.
	Terminal	Indicates the beginning and ending of a program or sub process.
	Process	Represents a set of operations that changes value, form, or location of data.
	Decision	Shows a conditional operation that determines which one of the two paths the program will take.
	Input/Output	Indicates the process of inputting and outputting data.

FLOWCHART SYMBOLS 2/2

Shape	Name	Description
	Predefined Process/ A Black Box	Shows named process which is defined elsewhere.
	(On-page) Connector	Pairs of labeled connectors replace long or confusing lines on a flowchart page.
	(Off-page) Connector	A labeled connector for use when the target is on another page.
	Data File or Database	Data represented by a cylinder symbolizing disk drive/any kind...

FLOW DIAGRAM — C-STYLE FOR LOOP



```
for (i = 0; i < n; i++)  
{  
    printf("*");  
}
```



BULLET POINT DESCRIPTION

Input:

N: Number of elements in the array, integer

Arr: Array of double type

Output:

Maximum: Variable of double type

Method:

1. Initialize Maximum to Arr[0]. Set Index to 0.
2. Increment Index by 1.
3. If Index < N, go to step 4. Otherwise go to step 6.
4. If Maximum < Arr[Index], update Maximum to Arr[Index].
5. Go back to step 2.
6. Print Maximum and end.

PSEUDOCODE — PASCAL STYLE

```
function factorial(n: integer): integer;  
begin  
    if n = 0 then  
        factorial := 1  
    else  
        factorial := n * factorial(n - 1);  
end;
```

When the C language was being born, Pascal was already a popular language. It contributed significantly to the world of programming, but it did not stand the test of time. Languages derived from C eventually replaced it entirely.

Advantages and Disadvantages — Flowchart

Advantages:

- ✂ **Visualization** — presents the algorithm graphically, easier to understand for visual learners
- ✂ **Intuitiveness** — arrows and shapes clearly show the sequence of actions
- ✂ **Universality** — can represent various types of algorithms regardless of programming language

Disadvantages:

- ✂ **Complexity** — for large algorithms the diagram can become unreadable
- ✂ **Time-consuming** — creating a detailed flowchart takes significant effort
- ✂ **Limitations** — not always easy to represent abstract concepts

Advantages and Disadvantages — Bullet Point Description

Advantages:

- ✂ **Simplicity** — easy to understand, even for beginners
- ✂ **Flexibility** — natural language allows for more detailed explanations
- ✂ **Focus on the idea** — allows focus on the main idea rather than implementation details

Disadvantages:

- ✂ **Lack of formality** — not as precise as pseudocode or a flowchart
- ✂ **Potential ambiguity** — interpretation may vary depending on the reader
- ✂ **Limitations for complex algorithms** — a bullet point list may be insufficient

Advantages and Disadvantages — Pseudocode

Advantages:

- ✂ **Intermediate** between natural language and programming language — easier to understand than pure code, more formal than bullet points
- ✂ **Precision** — allows for a precise description of the algorithm's steps
- ✂ **Language independence** — can be easily translated into any programming language

Disadvantages:

- ✂ **Less intuitive than a flowchart** — requires more effort to understand the algorithm's logic
- ✂ **Possibility of errors** — if not written carefully, may contain inaccuracies

PSEUDOCODE CONSTRUCTS

- ✂ **Expressions:** Assignment uses `←` (= in C++). Equality uses `=` (== in C++).
- ✂ **Method declarations:** `Algorithm name(param1, param2, ...)` — declares a method.
- ✂ **Decision structures:** `if condition then true-actions [else false-actions]` . Indentation indicates scope.
- ✂ **While-loops:** `while condition do actions` .
- ✂ **Repeat-loops:** `repeat actions until condition` .
- ✂ **For-loops:** `for variable-increment-definition do actions` .
- ✂ **Array indexing:** `A[i]` — indexed from 1 to n (math) or 0 to n-1 (C/C++/Java).
- ✂ **Method calls:** `object.method(args)` .
- ✂ **Method returns:** `return value` .

THE RAM MODEL

Primitive Operations

THE RAM MODEL — MOTIVATION

If we wish to analyze a particular algorithm without performing experiments on its running time, we can take an **analytic approach** directly on the high-level code or pseudocode.

We define a set of high-level **primitive operations** that are largely independent from the programming language used and can be identified also in the pseudocode.

THE SEVEN PRIMITIVE OPERATIONS

Symbol	Operation	Example
c_1	Assigning a value to a variable	<code>x = 5</code>
c_2	Calling a method / function	<code>sort(arr)</code>
c_3	Performing an arithmetic operation	<code>a + b</code> , <code>i++</code>
c_4	Comparing two numbers	<code>a > b</code> , <code>a == b</code>
c_5	Indexing into an array	<code>arr[i]</code>
c_6	Following an object reference	<code>node->next</code>
c_7	Returning from a method	<code>return x</code>

THE RAM MODEL — DEFINITION

This approach gives rise to the **Random Access Machine (RAM)** model — which should **not** be confused with "random access memory." It views a computer as a CPU connected to a bank of memory cells.

The term ***random access*** refers to the ability of the CPU to access an arbitrary memory cell with **one primitive operation**.

We assume the CPU can perform any primitive operation in a **constant number of steps** which do not depend on the size of the input. Thus, the number of primitive operations an algorithm performs corresponds directly to its running time in the RAM model.

WHY THESE SEVEN? — WHY CONSTANT TIME?

Each primitive operation takes a **fixed, constant amount of time** on any real computer. It does not matter whether:

- ✂ the variable holds the value 1 or 1,000,000
- ✂ the array has 10 elements or 10 million elements
- ✂ the comparison is true or false

The implicit assumption is that the running times of different primitive operations will be **fairly similar**. Thus, the total number t of primitive operations will be **proportional** to the actual running time.

This is why we can count them and use the count as a proxy for running time.

Each Operation — One Step

```
int x = 5;
```



Step	Operation	Cost
	assign 5 to x	c_1
		1

```
int result = a + b;
```



Step	Operation	Cost
	perform a + b	c_3
		1
	assign to result	c_1
		1
Total		2

ARRAY ACCESS 1/2

```
if(a > b)
```



Step	Operation	Cost
compare a and b	c_4	1

```
int x = arr[i];
```



Step	Operation	Cost
index arr[i]	c_5	1
assign to x	c_1	1
Total		2

ARRAY ACCESS 2/2

```
int x = arr[i] + arr[j];
```



Step	Operation	Cost
index arr[i]	c_5	1
index arr[j]	c_5	1
perform addition	c_3	1
assign to x	c_1	1
Total		4

FUNCTION CALL AND RETURN

```
int add(int a, int b)
{
    return a + b;
}

int result = add(x, y);
```



Step	Operation	Cost
call add(x, y)	c_2	1
perform a + b	c_3	1
return from add	c_7	1
assign result	c_1	1
Total		4

COUNTING OPERATIONS

in a Loop

A SIMPLE LOOP

$$T(n) = 1 + 1 + (n + 1) + 2n + n + n + n + 1 = 6n + 4$$

```
int sum = 0;
for (int i = 0; i < n; i++)
    sum = sum + arr[i];
return sum;
```



Line	Operation	Count
int sum = 0	c_1	1
int i = 0	c_1	1
i < n	check c_4	$n + 1$
i++	$c_3 + c_1$	$2n$
arr[i]	c_5	n
sum + arr[i]	c_3	n
sum = ...	c_1	n
return sum	c_7	1

A NESTED LOOP

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    sum = sum + arr[i] + arr[j]; // c5 + c5 + c3 + c3 + c1 = 5 ops
```



Operation	Per inner iteration	Total
<code>arr[i]</code> — c_5	1	n^2
<code>arr[j]</code> — c_5	1	n^2
<code>first +</code> — c_3	1	n^2
<code>second +</code> — c_3	1	n^2

FROM COUNT

to $T(n)$

THE PATTERN

Algorithm	Exact $T(n)$	Dominant term
Single statement	constant	$\Theta(1)$
Single loop	$6n + 4$	$\Theta(n)$
Nested loop (2×)	$7n^2 + 3n + 4$	$\Theta(n^2)$
Triple nested loop	$an^3 + bn^2 + cn + d$	$\Theta(n^3)$
Halving loop	$a \cdot \log n + b$	$\Theta(\log n)$
Loop + halving	$a \cdot n \log n + b$	$\Theta(n \log n)$

WHY LOWER TERMS DISAPPEAR

$$T(n) = 7n^2 + 3n + 4$$

n	$7n^2$	$3n$	4	% that is NOT $7n^2$
10	700	30	4	4.6%
100	70,000	300	4	0.4%
1,000	7,000,000	3,000	4	0.04%
10,000	700,000,000	30,000	4	0.004%

As $n \rightarrow \infty$, the $3n$ and 4 terms **vanish** relative to $7n^2$.

CONSTANT COEFFICIENTS ALSO DISAPPEAR

$$T_A(n) = 7n^2 \quad T_B(n) = 2n^2$$

Both grow as n^2 . The constant factor (7 vs 2) depends on:

- ⌘ how many primitive operations we counted per step
- ⌘ the speed of the specific hardware
- ⌘ the quality of the compiler

These are **not properties of the algorithm** — they are properties of the environment. We care about the **shape** of growth, not the constant multiplier.

THE RULE — DROP EVERYTHING EXCEPT THE DOMINANT TERM

$$T(n) = 7n^2 + 3n + 4$$

Step 1: Keep only the dominant term: $7n^2$

Step 2: Drop the constant coefficient: n^2

Step 3: Express as asymptotic class: $T(n) = \Theta(n^2)$

This works because for large n :

$$\frac{1}{2}n^2 \leq 7n^2 + 3n + 4 \leq 8n^2 \quad \text{for all } n \geq n_0$$

which is exactly the Θ definition with $c_1 = 1/2$, $c_2 = 8$.

FROM $T(N)$

to Θ , O , Ω

CONNECTING $T(N)$ TO THE THREE NOTATIONS

Given $T(n) = 7n^2 + 3n + 4$:

Notation	Meaning	Statement
$\Theta(n^2)$	tight bound — grows exactly like n^2	$T(n) = \Theta(n^2)$
$O(n^2)$	upper bound — grows no faster than n^2	$T(n) = O(n^2)$
$\Omega(n^2)$	lower bound — grows no slower than n^2	$T(n) = \Omega(n^2)$

✂ Θ is the most informative — bounds both above and below

✂ O is most commonly used in practice — gives a guarantee

THE FULL WORKFLOW

1. Write the algorithm
↓
2. Count primitive operations ($c_1 \dots c_7$)
↓
3. Express as $T(n)$ — exact polynomial
↓
4. Keep only the dominant term
↓
5. Drop constant coefficients
↓
6. Express as $\Theta(f(n))$, $O(f(n))$, or $\Omega(f(n))$

ARRAY INDEXING

The AGU Rule

ARRAY INDEXING — THE AGU RULE

Modern processors have a dedicated **Address Generation Unit (AGU)** — a separate hardware unit that computes memory addresses independently from the ALU. This means that a simple array offset does **not** consume an arithmetic operation.

ARRAY INDEXING — SIMPLE INDEX

When the index is a **simple variable or constant** — the AGU handles it in one step:

```
tab[i] // c_5 — AGU computes base + i*sizeof(type)
tab[0] // c_5 — constant offset, AGU handles it
*tab   // c_6 — pointer dereference, same mechanism
```



`tab[i]` and `*(tab+i)` compile to the **same machine instruction**.

The offset is built into the instruction — no separate ALU operation.

ARRAY INDEXING — COMPOUND EXPRESSION

When the index contains **any compound expression**, the ALU must compute it **before** the AGU can perform the access:

```
tab[i+j]  // c_3 + c_5 → 2 ops (ALU: i+j, then AGU)
tab[i-1]  // c_3 + c_5 → 2 ops (ALU: i-1, then AGU)
tab[i*2]  // c_3 + c_5 → 2 ops (ALU: i*2, then AGU)
tab[i+j+1] // c_3 + c_3 + c_5 → 3 ops
```



We count the inner expression first — then add c_5 for the indexing.
We do **not** assume compiler or CPU optimizations.

ARRAY INDEXING — THE RULE

Expression	Rule	Cost
<code>tab[i]</code>	simple variable — AGU only	$c_5 = 1$
<code>tab[0]</code>	constant — AGU only	$c_5 = 1$
<code>*tab</code>	pointer dereference — AGU	$c_6 = 1$
<code>node->next</code>	object reference — AGU	$c_6 = 1$
<code>tab[n-1]</code>	compound expression + index	$c_3 + c_5 = 2$
<code>tab[i+j]</code>	compound expression + index	$c_3 + c_5 = 2$
<code>tab[i*2]</code>	compound expression + index	$c_3 + c_5 = 2$
<code>tab[i+j+1]</code>	two expressions + index	$c_3 + c_3 + c_5 = 3$

ARRAY INDEXING — WHY THIS RULE?

You might ask: if the processor can sometimes handle `tab[i*2]` in a single instruction — why do we count it as two operations?

The answer is the **RAM model**. We analyze the algorithm from the **source code**, not from the generated assembly. We do not know what compiler is used, what optimizations are applied, or what the target architecture is.

So we apply the **conservative rule**: any expression inside `[ ]` that is not a simple variable or constant — count it separately as `c_3`.

This gives us counts that are **consistent, predictable, and machine-independent** — which is exactly the purpose of the RAM model.

ALGORITHM CORRECTNESS

arrayMax

SEARCHING FOR THE MAXIMUM VALUE IN AN ARRAY

```
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
    {
        if (currentMax < A[i])
            currentMax = A[i];
    }
    return currentMax;
}
```



Algorithm arrayMax(A, n):

Input: array A, $n \geq 1$ integers

Output: maximum element in A

```
currentMax ← A[0]
for i ← 1 to n - 1 do
    if currentMax < A[i] then
        currentMax ← A[i]
return currentMax
```

CURIO — TWO EQUIVALENT IMPLEMENTATIONS

```
// Version 1 — if statement
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
    {
        if (currentMax < A[i])
            currentMax = A[i];
    }
    return currentMax;
}
```



```
// Version 2 — ternary operator
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
        currentMax =
            currentMax < A[i]
            ? A[i] : currentMax;
    return currentMax;
}
```



Which is faster? We will answer this after counting primitive operations.

USING PSEUDOCODE TO PROVE ALGORITHM CORRECTNESS

By inspecting the pseudocode, we can argue about the correctness of `arrayMax` with a simple argument.

`currentMax` starts equal to the first element of A. We claim that **at the beginning of the i -th iteration**, `currentMax` equals the maximum of the first i elements.

Since we compare `currentMax` to `A[i]` in iteration i — if this claim is true before the iteration, it will be true after it for $i+1$. Thus, after $n-1$ iterations, `currentMax` equals the maximum element in A.

COUNTING PRIMITIVE OPERATIONS — ARRAYMAX

```
currentMax ← A[0]    → c_5 + c_1 = 2 (once)
i ← 1                → c_1 = 1 (once)
i < n check          → c_4 = n (n times)
A[i] vs currentMax   → c_5 + c_4 = 2(n-1)
i++                  → c_3 + c_1 = 2(n-1)
currentMax ← A[i]    → c_5 + c_1 = 2(n-1) (worst)
return currentMax    → c_7 = 1 (once)
```

Worst case (sorted ascending):

$$2 + 1 + n + (n - 1)(6) + 1 = 7n - 2$$

Best case (A[0] is maximum):

$$2 + 1 + n + (n - 1)(4) + 1 = 5n$$

FINAL COUNT — $T(N)$ FOR ARRAYMAX

Best case — $A[0]$ is the maximum,
`currentMax` never reassigned:

$$T(n) = 5n$$

Worst case — array sorted ascending,
`currentMax` reassigned at every step:

$$T(n) = 7n - 2$$

$$5n \leq T(n) \leq 7n - 2$$



$$T(n) = \Theta(n)$$

The exact constants (5 and 7) are irrelevant for the asymptotic class.
Both best and worst case are **linear**.

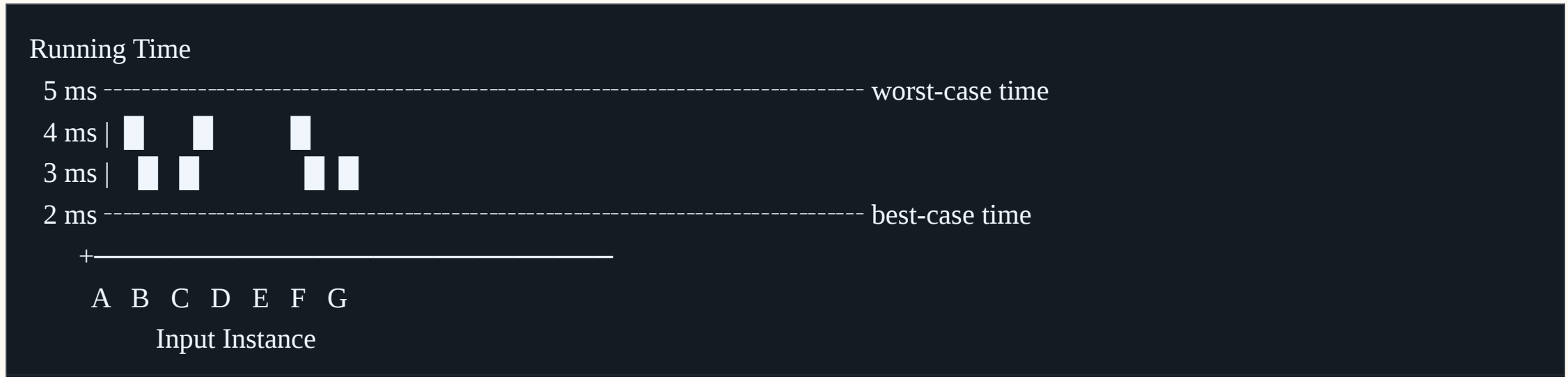
BEST CASE, WORST CASE

Average Case

AVERAGE-CASE AND WORST-CASE ANALYSIS

Like `arrayMax`, an algorithm may run faster on some inputs than others. We may wish to express running time as an **average** taken over all possible inputs.

Although such an **average-case analysis** would often be valuable, it is typically quite challenging. It requires us to define a **probability distribution** on the set of inputs.



WHY WE USE WORST-CASE ANALYSIS

- ✖ Average-case analysis requires calculating **expected running times** based on a given input distribution — heavy mathematics and probability theory.
- ✖ Therefore, we typically characterize running times in terms of the **worst case**.
`arrayMax` executes $T(n) = 7n - 2$ primitive operations in the worst case.
- ✖ This is much **easier** — requires only identifying the worst-case input, which is often straightforward.
- ✖ Designing for the worst case leads to stronger algorithmic "muscles" — **much like a track star who always practices by running uphill.**

WHICH IS FASTER? — BACK TO THE CURIO

```
// Version 1 — if statement
if (currentMax < A[i])
    currentMax = A[i];

// Version 2 — ternary operator
currentMax = currentMax < A[i] ? A[i] : currentMax;
```



In the **worst case** both versions execute the same primitive operations.

In the **best case** Version 1 skips the assignment when $A[i] \leq \text{currentMax}$. Version 2 **always** performs the assignment.

In the RAM model both are $\Theta(n)$. The difference is only in the constant — which disappears asymptotically.

QUESTIONS?

THANK

You